JWT vs HttpOnly cookie

1. JWT in header

Scenario tipico:

- Login con `POST /login` → il server genera un **JWT** (spesso firmato con HS256)
- Il client (SPA/JS) salva il token in:
    - `localStorage` **oppure**
    - `sessionStorage`
- Ad ogni richiesta → header:
  `Authorization: Bearer <jwt>`

Problema principale

Se il sito ha anche solo una **vulnerabilità XSS**, uno script malevolo può fare:

localStorage.getItem("token") // oppure sessionStorage

e rubarsi il JWT.
A quel punto il token è **usabile ovunque**, da chiunque lo possieda.

In più spesso:

- JWT molto lunghi
- scadenza troppo ampia
- troppe informazioni dentro il payload
- nessun meccanismo di **rotazione** / revoca

Risultato: se il token viene rubato, l'attaccante è loggato "per sempre" finché il token non scade.

2. Cosa cambia con HttpOnly cookie

L'idea moderna è:

1. Il server, dopo il login, **NON** ti dà il JWT da salvare nel JS.

Invece, risponde con un **Set-Cookie**:

```
Set-Cookie: access_token=<jwt-o-id-sessione>;
        HttpOnly;
        Secure;
        SameSite=Strict;
        Path=/
```

2.
3. Il browser:
    - salva il cookie
    - lo **rimanda automaticamente** in ogni richiesta verso quel dominio.

Tu dal frontend non fai più:
`Authorization: Bearer ...`
ma semplicemente chiami le API (fetch/axios) e il browser allega i cookie.

Perché è più sicuro?

Flag **HttpOnly** → il cookie **NON** è leggibile da JavaScript:

```
document.cookie  // non mostra il cookie HttpOnly
```

- Quindi, anche se hai XSS, l'attaccante non può rubare il token semplicemente leggendolo dal client.
- Flag **Secure** → il cookie viaggia solo su HTTPS (non in chiaro).
- Flag **SameSite**:
    - `Strict` o `Lax` riducono i rischi di **CSRF**, cioè richieste da altri siti che usano il tuo cookie.

3. Ma allora JWT sono “insicuri”?

Non è che JWT siano insicuri in sé.
Problemi comuni:

1. **Uso improprio**:
    - Token lunghissimi, con un sacco di dati sensibili.
    - Nessuna rotazione.
    - Nessuna revoca server-side (“se ho un JWT valido posso bypassare tutto”).
2. **Storage insicuro** nel frontend:
    - `localStorage` + XSS = furto del token.
3. **Assenza di controlli extra**:
    - niente IP binding, niente device binding, niente controllo su user-agent.

Spesso oggi si consiglia di:

- usare JWT come **access token** a breve durata
- tenerli **nel backend o in cookie HttpOnly**, non nel JS leggibile.

4. Flusso moderno tipico (semplificato) per le tue lezioni

5. Confronto didattico: Header JWT vs HttpOnly cookie

Puoi sintetizzare così ai ragazzi:

Aspetto	JWT in header + localStorage	JWT/Session in HttpOnly Cookie
Lettura JS	✅ sì (`localStorage`)	❌ no (HttpOnly)
Rischio XSS	**alto** (token rubabile)	più basso (token non leggibile da JS)
Rischio CSRF	basso (token in header manuale)	più alto se SameSite è `None`

Comodità	client controlla header	browser gestisce i cookie da solo

Messaggio importante da passare:

> Non è che “JWT sono il male” e “cookie sono il bene”.
> Il tema è **dove** tieni il token e come gestisci **XSS/CSRF**.

---

1. Implementre Http CLient Only

Da terminale:

```bash
npm install cookie-parser
```

---

3. Modifica di `server.js` per usare i cookie

In `server.js`, aggiungi `cookie-parser` e (se usi una SPA) abilita CORS con credenziali.

server.js aggiornato (versione semplificata)

```
// agganciamo le librerie che abbiamo installato con node
const express = require('express');
const sqlite3 = require('sqlite3').verbose();
const bodyParser = require('body-parser');
const cors = require('cors');
const cookieParser = require('cookie-parser'); // <--- NEW

// dichiariamo su che porta risponderà il server web
```

```
const port = 3000;
//creiamo l’istanza del server
const app = express();

// Middleware
// Se usi una SPA su un’altra porta (es. Angular su 4200), meglio così:
app.use(cors({
  origin: 'http://localhost:4200', // oppure l'origin del tuo frontend
  credentials: true               // <--- per mandare i cookie
}));

app.use(bodyParser.json());
app.use(cookieParser()); // <--- abilita lettura cookie HttpOnly

// Importa le rotte
const coursesRoutes  = require('./routes/courses');
const studentsRoutes = require('./routes/students');
const teachersRoutes = require('./routes/teachers');
const authRoutes     = require('./routes/auth');

// Usa le rotte
app.use('/api/v1/courses', coursesRoutes);
app.use('/api/v1/students', studentsRoutes);
app.use('/api/v1/teachers', teachersRoutes);
app.use('/api/v1/login', authRoutes);
```

```
// Avviare il server
app.listen(port, () => {
  console.log(`Server in ascolto sulla porta ${port}`);
});
```

Così il server è pronto a:

- **leggere** cookie (`req.cookies`)
- permettere al browser di **inviare** cookie alle API (`credentials: true` lato CORS, e `credentials: 'include'` lato fetch/axios).

---

4. Modifica della rotta di login per usare HttpOnly cookie

Ora modifichiamo `routes/auth.js`: invece di mandare il token nel JSON, lo mettiamo in un cookie HttpOnly.

routes/auth.js aggiornato

```
const express = require('express');
const router = express.Router();
const jwt = require('jsonwebtoken');
const db = require('../db/connection');

const SECRET_KEY = 'ChiaveComplessa!';

// POST /api/v1/login
router.post('/', (req, res) => {
  const { username, password } = req.body;
```

```
  db.get(
    'SELECT * FROM docente WHERE username = ? AND password = ?',
    [username, password],
    (err, results) => {
      if (err) {
        console.error(err);
        return res.status(500).send('Errore nel database');
      }

      if (!results) {
        return res.status(404).json({ message: 'Username o password errati!!' });
      }

      // payload minimale
      const docentePayload = {
        cognome: results.cognome,
        nome: results.nome,
        materie: results.materie
      };

      // genera JWT
      jwt.sign(docentePayload, SECRET_KEY, { expiresIn: '1h' }, (err, token) => {
        if (err) {
          return res.status(500).json({ message: 'Errore nella creazione del token'
});
        }
```

```
        // *** QUI CAMBIA: settiamo un cookie HttpOnly ***
        res.cookie('authToken', token, {
          httpOnly: true,   // non leggibile da JS
          secure: false,    // in sviluppo su http://localhost deve essere false
          sameSite: 'lax',  // protegge un minimo da CSRF ed è ok per molte SPA
          maxAge: 60 * 60 * 1000 // 1h in millisecondi
        });

        // Nel body NON mandiamo il token
        res.status(200).json({
          message: 'login ok',
          docente: docentePayload
        });
      });
    }
  );
});

module.exports = router;
```

In produzione con HTTPS: basta mettere `secure: true`.
Su localhost HTTP: **deve rimanere `secure: false`**, altrimenti il browser non manda il cookie.

---

5. Middleware per verificare il JWT dal cookie

Ora ci serve un middleware riusabile per proteggere le rotte (es. `/courses`).

Crea un file, per esempio: `middlewares/checkAuth.js`:

```
// middlewares/checkAuth.js
const jwt = require('jsonwebtoken');

const SECRET_KEY = 'ChiaveComplessa!';

function checkAuth(req, res, next) {
  // leggiamo il cookie impostato nel login
  const token = req.cookies.authToken;

  if (!token) {
    return res.status(401).json({ message: 'Token mancante. Effettua il login.' });
  }

  jwt.verify(token, SECRET_KEY, (err, decoded) => {
    if (err) {
      console.error('Errore verifica token:', err);
      return res.status(401).json({ message: 'Token non valido o scaduto.' });
    }

    // metto i dati del docente nella request, così le rotte li possono usare
    req.user = decoded;
    next();
  });
}
```

```
module.exports = checkAuth;
```

---

6. Proteggere le rotte (esempio su `courses.js`)

Nel tuo `routes/courses.js` ora puoi usare il middleware per fare in modo che tutte le operazioni sui corsi richiedano il login.

routes/courses.js aggiornato (parte iniziale)

```
const express = require('express');

const router = express.Router();

const db = require('../db/connection');

const checkAuth = require('../middlewares/checkAuth'); // <--- NEW


// Tutte le rotte sotto richiedono autenticazione

router.use(checkAuth);


/**********  COURSES ***************/


// searchAll

router.get('/', (req, res) => {

  db.all('SELECT * FROM corso', (err, results) => {

    if (err) {

      console.error(err);

      res.status(500).send('Errore nel database');

    } else {

      res.json(results);

    }
```

```
  });
});

// ... il resto delle rotte (GET /:id, POST, PUT, DELETE) rimangono uguali
```

Oppure, se vuoi che **solo alcune** rotte siano protette, puoi fare:

```
router.get('/', checkAuth, (req, res) => { ... });
router.post('/', checkAuth, (req, res) => { ... });
```

---

7. Logout: come cancellare il cookie

Ti conviene definire una rotta, per esempio in `routes/auth.js`:

```
// POST /api/v1/login/logout
router.post('/logout', (req, res) => {
  res.clearCookie('authToken', {
    httpOnly: true,
    secure: false,  // come sopra: false in dev, true in produzione
    sameSite: 'lax'
  });
  res.status(200).json({ message: 'Logout effettuato' });
});
```

E in `server.js` non devi cambiare nulla: è sempre sotto `/api/v1/login`.

---

8. E lato frontend?

Due note veloci che puoi usare anche con i ragazzi:

- Con `fetch`:

```
fetch('http://localhost:3000/api/v1/courses', {
  method: 'GET',
  credentials: 'include' // <--- per mandare i cookie
});
```

- Con Axios:

```
axios.get('http://localhost:3000/api/v1/courses', {
  withCredentials: true
});
```

---

9. Risposta secca alla tua domanda

“Server in locale senza HTTPS, con HttpOnly cookie funziona lo stesso?”

- **Sì**, funziona:
    - `HttpOnly` funziona anche su HTTP → il cookie non è leggibile da JS.
    - Il vantaggio contro XSS c’è comunque.
- L’unica cosa: **non** puoi usare `secure: true` (cookie inviato solo su HTTPS), quindi in sviluppo usi `secure: false`.
- In produzione è caldamente consigliato:
    - HTTPS
    - `secure: true`

    - `sameSite` impostato correttamente (es. `lax` o `strict` a seconda dei casi).

Se vuoi, nel prossimo passo possiamo:

- aggiungere anche un **refresh token** con secondo cookie
- o preparare una slide per i ragazzi con il confronto “prima/dopo” e spiegazioni XSS/CSRF.